ASC 2025 Proposal of XTU

Team members:
Tutor:

CONTENTS

1. Supercomputing Activities 3
1.1. The HPC Platform of XTU 3
1.2. HPC Activities 3
1.3. Campus Activities 4
1.4. Research and Applications 4
1.5. Key Achievements 5
2. Our Big Family 6
2.1. Introduction of members 6
2.2. Team slogan 6
3. Design of our HPC System 7
3.1. Cluster overview 7
3.2. Hardware Configurations 7
3.3. Software Components 7
3.4. Power Description 7
3.5. Performance Evaluation 7
4. HPL Test and Optimization 8
4.1. Performance Optimation Methods 8
4.2. Machine Performance Analysis 10
4.3. Experimental Results 10
4.4. Summary 11
5. HPCG Test and Optimization 12
5.1. Performance optimation Methods 12
5.2. Experimental Results 12
5.3. Summary 12
6. Optimization for AlphaFold3 Inference 13
6.1. Introduction 13
6.2. Related work 13
6.3. Proposed Methods 13
6.4. Experiments 13
6.5. Conclusion 13

7. RNA m^5C Modification Site Detection and Performance Optimization 14
7.1. Introduction 14
7.2. Related Work 14
7.3. Proposed Methods 14
7.4. Experiments 14
7.5. Conclusion 14
Acknowledge 14
Reference 14
References 15

1. Supercomputing Activities

Xiangtan University(XTU) is a prestigious comprehensive national key university and a "Double First-Class" construction university of the country. Standing majestically in the hometown of the great leader Mao Zedong, XTU is a place of great renown. XTU not only has modern teaching facilities, but also is equipped with an advanced high-performance computing center. In terms of high-performance talent cultivation, the school provides students with abundant high-performance learning resources, including cutting-edge professional courses, high-level academic lectures, innovative research projects, and a powerful high-performance computing platforms. These resources not only broaden students' horizons, but also provide them solid academic and technical support. After years of meticulous cultivation, the student teams of XTU participate in the ASC competition for the first time this year. Next, we will introduce the university's supercomputing platform and supercomputing-related activities.

1.1. **The HPC Platform of XTU.**

Xiangtan University's Super Heterogeneous Parallel Computing Cluster is a key project supported by the provincial finance under the "14th Five-Year Plan" for new infrastructure construction. The cluster includes ClusterEngine high-performance computing platform and AIstation artificial intelligence platform, and is open to all teachers and students of the university. The computing capacity of the cluster is the theoretical computing capacity of 509 trillion times of CPU double-precision peak performance (6,112 physical cores) and 1552 trillion times of GPU single-precision theoretical computing capacity. The overall logical framework design diagram of the cluster is shown in the figure.

The ClusterEngine high-performance computing platform consists of 1 management node, 1 login node, 1 physical monitoring platform, 85 general-purpose computing nodes, 2 large-memory computing nodes, totaling 5472 CPU cores and a total memory capacity of 44.5TB. The nodes are interconnected through Gigabit Ethernet and Infiniband. The management and login nodes are each equipped with 16 CPU cores and 64GB of memory.

The cluster has two resource partitions: CU and FAT partitions. The CU partition is configured with 80 general-purpose computing nodes, each with 64 Intel 8358 CPU cores and 512GB of memory, totaling 5,120 CPU cores. The FAT partition is configured with 2 large-memory computing nodes, each with 16 Intel 6334 CPU cores and 1TB of memory.

The main function of the platform is to provide high-performance scientific and engineering computing services to the entire university, meeting the needs of various disciplines for large-scale data processing and large-scale scientific computing. The platform's main application areas include providing high-performance computing environments for mathematics, statistics, mechanics, physics, chemistry, chemical engineering, biology, earth science, aerospace science and technology, computer science, and many other disciplines. The platform supports a wide range of application software, such as MATLAB, OpenFOAM, ABINIT,GROMACS, NAMD and so on.

The AIstation artificial intelligence platform consists of 10 GPU computing nodes, with a total of 640 CPU cores, 80 NVIDIA A100 acceleration cards, a total memory capacity of 5TB, and shared with the parallel storage and high-performance computing platform. The total capacity is 1.3PB. The GPU's measured peak speed can reach 888.9 trillion operations per second. This platform supports single-machine and distributed running of mainstream AI frameworks as well as task visualization.

1.2. **HPC Activities.**

Xiangtan University actively conducts academic activities and research projects related to supercomputing. For instance, in June 2022, Professor Peng Shaoliang, the deputy director of the National Supercomputing Changsha Center, delivered an academic report titled ”Prospects of Computational Power and Applications of the Metaverse Based on Supercomputing” at Xiangtan University. Moreover, the university has participated in the construction of the Hunan Computing Power Network Integration Collaborative Innovation Platform, promoting the interconnection and networking of computing infrastructure and the efficient utilization of computing resources. Through these supercomputing platforms and related activities, Xiangtan University not only provides students with abundant learning resources but also helps the university to carry out scientific research innovation and talent cultivation in the field of high-performance computing.

1.3. **Campus Activities.**

To cultivate talents in the field of high-performance computing, the school provides various activities. In terms of courses, the relevant courses offered by the school include ”Parallel Programming”, ”Computer Organization Principles”, ”Syntax and Semantics of Compilers”, ”Computer System Architecture”, ”Operating Systems”, ”Principles of Computer Networks”, ”Linux System Analysis and Programming”, ”Algorithm Design and Analysis”, ”Mathematical Analysis”, ”C Programming”, ”Graph Theory and Combinatorial Mathematics”, ”Data Structures”, ”Artificial Intelligence”, etc., which are a wealth of courses. In terms of regular training, through platforms such as EasyHPC, the school invites previous ASC participants to provide regular practice and training for students. Separate classrooms and servers are also provided for students. In terms of interest groups, in addition to forming ASC competition teams, the school’s ACM team also actively participates.

1.4. **Research and Applications.**

The research and application achievements of XTU in the field of supercomputing are remarkable, including parallel solvers, the development of high performance computing software, the optimization of computer system architecture, the design of high performance computing algorithms and the application of high performance computing in scientific research, engineering design, industrial production and other fields. The following is only a brief list.

For example, the JXPAMG solver jointly developed with the Beijing Institute of Applied Physics and Computational Mathematics has become the default linear solver of JASMIN, a high-performance parallel computing software platform, with its algorithm innovation, efficient parallel performance and multi-field applicability, and its comprehensive performance is better than the international popular similar solvers. The solver has greatly improved the computational efficiency of some defense application model problems, and has an important influence in the field of scientific calculation and engineering simulation.

The chemical flooding model is widely used in the numerical simulation of complex reservoir because of its high oil recovery. Therefore, for the numerical simulation of chemical flooding reservoir, CPU-GPU heterogeneous parallel discretization technology is used to discretized the chemical flooding model. The numerical experiment results show that the CPU-GPU heterogeneous parallel discretization method has a good acceleration effect. For example, for the calculation example with a grid size of about 1.25 million, The speed of the new algorithm is 3.99 times faster than that of the serial algorithm. Then, a non-smooth aggregation algebraic multi-grid preconditioned conjugate gradient iterative algorithm (UA-AMG-CG) based on CPU-GPU heterogeneous system with low operator complexity, high acceleration ratio and good robustness is designed for water phase

pressure equation and component concentration equation, and the corresponding program module is developed.

1.5. **Key Achievements.**

As mentioned above, XTU's research in supercomputing includes many aspects, and have published multiple papers in international journals and conferences in the fields of mathematics and computer science and so on. Such as *CCF Transactions on High Performance Computing, Applicable Analysis, Mathematical Methods in the Applied Sciences, International Conference on Management of Data(SIGMOD) International Conference on Very Large Data Bases(VLDB), IEEE Transactions on Parallel and Distributed Systems(TPDS)* and other journal conferences, here is a brief overview of two of the outcomes:

Innovation 1: Parallel reservoir simulators on Tianhe-2 [1].

Parallel reservoir simulators are now widely used with availability of super computers. Modern massively parallel supercomputers demonstrate great power for simulating large-scale reservoir models. However, improving scalability and efficiency for fully implicit methods on emerging parallel architectures is still challenging. In this work, we present a robust discretization together with a parallel linear solver algorithm; and we explore the parallel implementation on the world's fastest supercomputer Tianhe-2. Starting with a general compositional model, we focus on the black oil model and developed Parallel eXtension Framework for parallelizing the serial simulator. A parallel preconditioner based on fast auxiliary space preconditioning (FASP) is applied to solve the Jacobian system arising from the fully implicit discretization. The parallel simulator was validated using large-scale black oil benchmark problems, for which parallel scalabilities were tested. Giant reservoir models with over 100 million grid blocks have been simulated within a few minutes, and test the strong scalability of AMG solver with 1 billion unknown. We also demonstrate the parallelization and acceleration using Intel Xeon Phi coprocessors. In the end, the efficiency of the parallel simulator is illustrated by a giant reservoir using up to 10,000 cores, for which the CPU and communication time are summarized for the linear and nonlinear algorithms.

Innovation 2: Parallel reservoir simulators on Tianhe-2

2. Our Big Family

to do—-after 0207

2.1. **Introduction of members.**

2.2. **Team slogan.**

Unlimited Algorithms, Fearless Supercomputing, Victory Walks with Us.

3. Design of our HPC System

to do—-after 0207

3.1. **Cluster overview.**

3.2. **Hardware Configurations.**

3.3. **Software Components.**

3.4. **Power Description.**

3.5. **Performance Evaluation.**

4. HPL Test and Optimization

The High-Performance Linpack Benchmark (HPL) is a benchmark program that tests the floating-point performance of high-performance computing cluster systems. HPL evaluates the floating-point computing capability of high-performance computing clusters.Calculate the $1st$ column element of matrix A to get the $1st$ column element of L, and the generation gets the $2nd$ column element of U. The loop operation completes the LU decomposition of A. The triangulation of the matrix is calculated by the following formula:

$$\begin{cases} a_{kj} = u_{kj} + \sum_{m=1}^{k-1} l_{mm} u_{mj} & j = k, k+1, \cdots \\ a_{ik} = l_{ik} u_{kk} + \sum_{m=1}^{k-1} l_{mm} u_{mik} & i = k+1, k+2, \cdots \end{cases} \tag{4.1}$$

In the formula, k=2,3, after transformation, we have:

$$\begin{cases} \mathrm{u}_{1i} = \mathrm{a}_{\mathrm{li}}, \mathrm{i} = 1, 2 \cdots, \mathrm{n}, \mathrm{u}_{\mathrm{kj}} = \mathrm{a}_{\mathrm{kj}} - \sum_{\mathrm{m}=1}^{\mathrm{k}-1} l_{\mathrm{km}} \mathrm{u}_{\mathrm{mj}}, 2\mathrm{k} \\ l_{\mathrm{a}} = \mathrm{a}_{\mathrm{il}} / \mathrm{u}_{11}, \mathrm{i} = 2, \cdots, \mathrm{n}, l_{\mathrm{ik}} = \left(\mathrm{a}_{\mathrm{ik}} - \sum_{\mathrm{m}=1}^{\mathrm{k}-1} l_{\mathrm{m}} \mathrm{u}_{\mathrm{mk}} \right) / \mathrm{u}_{\mathrm{kk}}, 2 < \mathrm{k} < \mathrm{i} \end{cases} \tag{4.2}$$

$$\begin{cases} y_1 = b_1 \\ y_i = b_i - \sum_{j=1}^{i-1} l_{ij} y_j, i = 2, 3, \cdots \end{cases}$$

$$\begin{cases} x_n = y_n / u_{nn} \\ x_i = \left(y_i - \sum_{j=i+1}^{n} u_{ij} x_j \right) / u_{ii}, i = n-1, n-2, \cdots \end{cases}$$

Based on the above algorithm, we can conclude that HPL primarily relies on matrix operations such as multiplication, addition, and matrix inversion and multiplication. Therefore, optimization efforts should focus on enhancing the efficiency of these matrix operations.

4.1. **Performance Optimation Methods.**

4.1.1. *Modification of HPL.dat*

. In the data document hpl.dat of HPL, many parameters affect the test results of HPL, among which the most important parameters are the problem size N, the block size NB, and the process array configuration (P×Q). Their optimal values are usually determined by factors such as the structure of the CPU, and we can obtain their optimal values for HPL through multiple estimates and tests. The other parameters have a smaller impact on the results, and we can also determine their optimal values through partial fine-tuning.

(1) **Solving the size of the matrix Ns:** The larger the scale N of the matrix, the higher the proportion of effective computation and the higher the system's floating-point processing performance; however, at the same time, the increase in matrix scale N will lead to an increase in memory consumption. Once the actual memory space of the system is insufficient, using the cache will significantly reduce performance. Therefore, for general systems, it is necessary to try to increase the scale N of the matrix while ensuring that the system cache is not used. Considering that the operating system itself requires a certain amount of memory, in addition to matrix A (N×N), HPL also has other memory overheads, and communication also needs to occupy some cache (the specific size of which varies depending on different MPIs).Generally, matrix A occupying about 80% of the total system memory is optimal N×N×8 = total system memory × 80%. We made a series of experiences. From the chart above, We can conclude that the Ns on Machine should be about 86000.

(2) **Solving the size of the matrix block**(NBs)**:** The size of the calculation granularity NB very much affects the pros and cons of the calculation performance, The coefficient matrix is divided into cyclic blocks of NB× NB and assigned to each process Medium to medium processing, too coarse or too fine particles will affect performance.Because the communication between cluster nodes adopts the message passing method, the NB is too small to pass messages frequently between nodes, resulting in long communication time, short computing time, and low computing/communication performance ratio; The NB is too large, which can easily lead to uneven load, which will also increase communication overhead. Therefore, communication between and within nodes also requires trade-offs.The selection of NB value is mainly to obtain the optimal value through actual testing. According to the test, NB should not be too large or too small, generally below 512, generally a multiple of 64,After multiple rounds of testing, we can see from the icons of the NBs that the machine we used achieved the best Gflops when NBs=384.

(3) **Two-dimensional processor grid (P×Q):** (P * Q) is the size of your grid which is equal to the number of processors your cluster has. The P and Q product is which is basically the number of processors we have. When picking these two numbers, try to make your grid as "square" as possible in shape. The HPL website mentions that best practice is to have it close to being a "square", thus P and Q should be approximately equal, with Q slightly larger than P.
Two-dimensional processor grid (P×Q) should better meet the following requirements:
P×Q= the number of processes;
P×Q= the number of GPUs=the number of processes.
One process corresponds to one GPU, and a process can spawn multiple threads to saturate the CPU.
$P \leq Q$.
Through multiple tests, the program is run with the number of threads = 16, and it is found that when the values of P and Q are 4, 4 or 2, 8, the running results show better.

4.1.2. *Third-Party Library*
. There are two alternative third-party libraries: MPI library and BLAS library. The commonly used MPI libraries are MPICH, IMPI, and OpenMPI. The BLAS library includes CBLAS, MKL, OpenBLAS, etc. Most of the calculations on the CPU are performed by the BLAS library, so the BLAS library has a relatively large impact on the calculations. Therefore, choosing the right BLAS library is still very important. Based on the paper and the competition materials of previous years, we chose to initially select Openblas and MKL as basic linear algebra subroutines, and then select MPICH and Openmp as MPI libraries for testing. The results are shown below.

The alternative third-party libraries include the MPI libraries and BLAS libraries. The frequently used MPI library have MPICH, IMPI, OpenMPI, and so on. BLAS library includes Atlas, GotoBLAS, MKL, and so on. Most of the calculations on the CPU are completed by the BLAS libraries. Therefore, it is still quite important to choose a proper BLAS library. On the machine 1, MKL and Intel MPI are the best according to the experience last year. However, we add Machine 2 this year. So we choose the Openblas and MKL as the Basic Linear Algebra

Subprograms and choose the MPICH and Intel MPI as the MPI libraries to do the experience again. The result is shown below.

Preliminary comparison chart of different MPI libraries and BLAS libraries

As we can see, Openblas performs quite well on the machine 2 but performs badly on the machine 1. We found that the library in the experience is the Dynamic Link Library(DLL) by checking the

TABLE 1. *Machine hardware configuration*

Name	Model	Quantity
CPU	AMD	1
GPU	Nvidia	2
RAM	DDR	64G
Hard Disk	SSD	256
	HDD	1T

make file. By using DLL, the test can dynamically use the function by loading the file timely and can cause more cost. On the machine 2, it will cache the file on its memory automatically to speed up the program and that's why Openblas performs well on it. So we compared the DLL and Static Library.

after changing to a static library, the overall calculation peak has improved a lot

The performance of Machine 1 is much better when using the Openblas. And the Intel Compiler is always better than the GNU Compiler. We suspect that the reason why the MKL library performs poorly on the Machine 2 is the version of MKL we are using is not very optimized for the new CPU.

4.1.3. *Accelerate Procedures by Using Appropriate Compiler Options*
. The first step to achieving high performance is to take full advantage of compiler optimizations, which is the simplest way to improve the program's performance. Some simple optimization measures can be automatically given to the compiler to improve speed. We have compiled the following options:

4.1.4. *Multi-thread acceleration*
. Based on the HPL testing algorithm, the matrix is decomposed into LU decomposition by dividing it into blocks using multiple processes. Each process is responsible for a matrix block, and OpenBLAS indeed supports multi-threading acceleration, and the number of threads used can be controlled through environment variables. Through reasonable thread configuration and parallel computation, OpenBLAS can significantly improve the performance of linear algebra calculations, especially for large-scale matrix operations; at the same time, OpenMP also supports multithreading acceleration, which is particularly suitable for parallel computation on shared memory architectures. It is a parallel programming model for compiler instructions and runtime environments, allowing programs to execute tasks in parallel on multiple processor cores, thereby improving computational efficiency.

So we guess that if multithreading is used to accelerate matrix calculations, multithreading for computation can improve performance Enhance scalability, optimize memory and cache access, help with load balancing, and improve running efficiency

4.2. **Machine Performance Analysis.** According to the above configuration, we can calculate the theoretical peak performance for our machine, i.e.,

$$3.7GHz*16(cores/CPU)*1(CPUs)*16(floatingpointexecutionunit)=947.2GFlops \quad (4.3)$$

$$689GFlops(GPU)*1=689GFlops \quad (4.4)$$

4.3. **Experimental Results.**

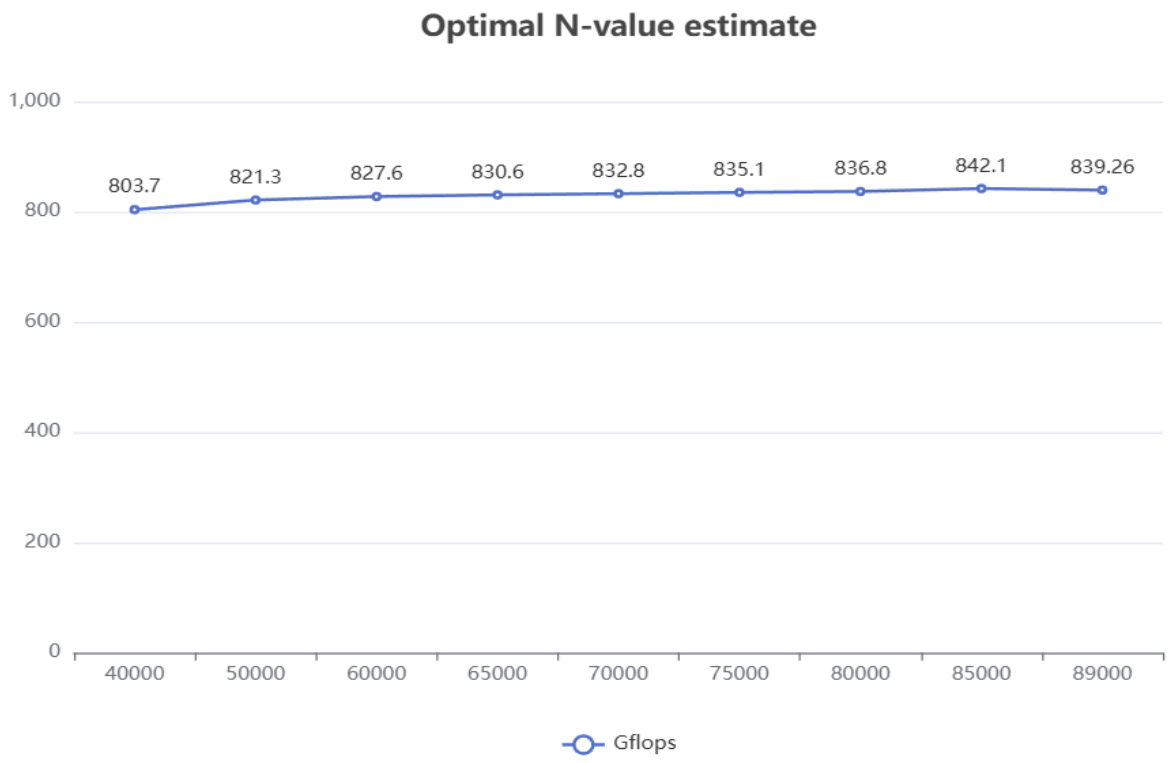


FIGURE 1. *Prediction of new individuals in multiple subpopulations*

4.3.1. *CPU Evaluation Results.*

4.3.2. *GPU Evaluation Results.*

4.4. **Summary.**

5. HPCG Test and Optimization

to do—-before 0123 close

The HPCG reference program simulates a three-dimensional thermodynamic motion problem, which can be abstracted to solve a discrete three-dimensional partial differential equation model problem. The large-scale positive and definite sparse matrix A is generated through the problem generation process. The form of the sparse matrix can consider various sparse forms according to the actual application, and generate vector B and initialize the value of solution X of the problem at the same time. The global problem size is $(nx \times npx) \times (ny \times npy) \times (nz \times npz)$, where $(nx \times ny \times nz)$ represents the size of subgrids in x, y and z dimensions of each subdomain, npx npy npz represents the number of subgrids in x, y and z dimensions respectively, and the computing task of each subgrid is assigned to a process. The size of the subgrid can be set by modifying the parameters in the HPCG.dat file or by the command line. After the problem is set up, HPCG will execute "N iterations" form times. Each iteration has the same initial value, and both values of M and N need to be large enough to test the system's recovery and continuous operation ability. At the end of the M iterations, numerical results can be compared to verify the correctness.

5.1. **Performance optimation Methods.**

5.1.1. *Modification of HPCG.dat.*

5.1.2. *Choose a Third-Party Library.*

5.1.3. *Using the Hybrid Version.*

5.1.4. *Others.*

5.2. **Experimental Results.**

5.2.1. *CPU Evaluation Results.*

5.2.2. *GPU Evaluation Results.*

5.3. **Summary.**

5.3.1. *CPU Version.*

5.3.2. *GPU Version.*

6. Optimization for AlphaFold3 Inference

to do—-about 0210 close

6.1. **Introduction.**

6.2. **Related work.**

6.3. **Proposed Methods.**

6.3.1. *GPU Optimization.*

6.3.2. *CPU Optimization.*

6.4. **Experiments.**

6.5. **Conclusion.**

7. RNA m^5C Modification Site Detection and Performance Optimization

to do—-about 0210 close

7.1. **Introduction.**

7.2. **Related Work.**

7.3. **Proposed Methods.**

7.3.1. *Detection Improvement.*

7.3.2. *Parallel Design.*

7.3.3. *Integrated Design.*

7.4. **Experiments.**

7.4.1. *Precision Evaluation.*

7.4.2. *Correlation Evaluation.*

7.4.3. *Performance Evaluation.*

7.4.4. *Scalability Evaluation.*

7.5. **Conclusion.**

Acknowledge

Reference

References

[1] W. Guan, C. Qiao, H. Zhang, C.-S. Zhang, M. Zhi, Z. Zhu, Z. Zheng, W. Ye, Y. Zhang, X. Hu *et al.*, "On robust and efficient parallel reservoir simulation on tianhe-2," in *SPE Reservoir Characterisation and Simulation Conference and Exhibition?* SPE, 2015, p. D031S017R004.